VERITAS: Web Vulnerability Evaluation and Real-time Intelligence Through Automated Simulation.

Supervisor: Prof. Madya Ts. Dr. Alya Geogiana Binti Buja

Table of Contents

2.1 Web Application Vulnerabilities and AI-Assisted Development

2.1.1 Overview of Web Application Vulnerabilities (OWASP Top Threats)

The Open Web Application Security Project (OWASP) is widely recognized as a primary reference for identifying and prioritizing security risks in web applications. OWASP focuses on the most critical threats by periodically updating and publishing the "OWASP Top 10" list, which identifies the ten most dangerous security vulnerabilities in web applications every three years (Shahid et al., 2022). This ranking enables security practitioners and development teams to align their mitigation strategies with the most significant and prevalent risks, thereby improving the overall resilience of web applications (Shahid et al., 2022).

The OWASP Top 10 is a frequently reviewed and updated document specifically designed to address the evolving landscape of web application security by highlighting the top ten critical vulnerabilities (Shahid et al., 2022). The current OWASP Top 10 (2025) list includes the following categories:

Vulnerabilities	CWEs Mapped	Max Incidence Rate	Total Occurrences	Total CVEs
Broken Access Control	40	20.15%	1,839,701	32,654
Cryptographic Failures	32	13.77%	1,665,348	2,185
Injection	37	13.77%	1,404,249	62,445
Insecure Design	39	22.18%	729,882	7,647
Security Misconfiguration	16	27.70%	719,084	1,375
Software Supply Chain Failures	6	9.56%	215,248	11
Authentication Failures	36	15.80%	1,120,673	7,147
Software and Data Integrity Failures	14	8.98%	501,327	3,331
Security Logging and Alerting Failures	5	11.33%	260,288	723
Mishandling of Exceptional Conditions	24	20.67%	769,581	3,416

Table 2.1: Incidence Rates and Occurrences of OWASP Top 10 Vulnerabilities

(Andrew van der Stock et al., 2025). These categories provide a structured taxonomy for understanding and addressing common weaknesses in modern web applications.

Empirical studies into web application vulnerabilities indicate that the frequency and diversity of security flaws have been steadily increasing over recent years (Rahman & Izurieta, 2022). Among the most commonly reported vulnerabilities are SQL injection, cross-site scripting (XSS), broken authentication, command-line injection, and identification and authentication failures (Rahman & Izurieta, 2022). These findings are consistent with the emphasis in the OWASP Top 10, as the results of that mapping study show that the most common vulnerabilities in web applications are largely covered by the OWASP Top 10 list (Rahman & Izurieta, 2022).

Furthermore, the mapping study reveals that SQL injection and cross-site scripting emerge as the most frequently detected vulnerabilities, reflecting their persistent exploitability and widespread impact on web systems (Rahman & Izurieta, 2022). The study also notes that Broken Access Control is mentioned in 38 different research papers, while Identification and Authentication Failures appear in 23 papers, underscoring the importance of access-control and authentication mechanisms in protecting web applications (Rahman & Izurieta, 2022). Together, these observations demonstrate that the OWASP Top 10 remains a relevant and practical framework for identifying, classifying, and mitigating the most critical web application vulnerabilities.

2.1.1.1 Broken Access Control

Prevalence and Evolution Broken Access Control has retained its #1 position in the OWASP Top 10 for 2025, with the highest incidence rate (3.74% average) and over 1.8 million occurrences across contributed data. Mapped CWEs, such as CWE-200 (Exposure of Sensitive Information) and CWE-918 (Server-Side Request Forgery), account for 32,654 CVEs, underscoring its persistence from prior editions(Andrew van der Stock et al., 2025).

Key Vulnerabilities Failures arise from violations like lacking least privilege (deny by default), URL tampering, insecure direct object references, and metadata manipulation (e.g., JWT tampering). Scenarios include SQL injection via unverified parameters, forced browsing to admin pages, and CORS misconfigurations enabling unauthorized API access.

Prevention Strategies Enforce server-side controls with deny-by-default policies, reusable mechanisms, and record ownership models; log failures and apply rate limiting. Short-lived JWTs or OAuth refresh tokens mitigate elevation risks, while testing should verify front-end and back-end enforcement.

Research Gaps and Implications While OWASP provides comprehensive mappings (40 CWEs), gaps exist in empirical studies on automated tooling impacts and emerging serverless contexts. Future theses could extend this by analysing Malaysian fintech applications, integrating cultural factors in access policy adoption(Andrew van der Stock et al., 2025).

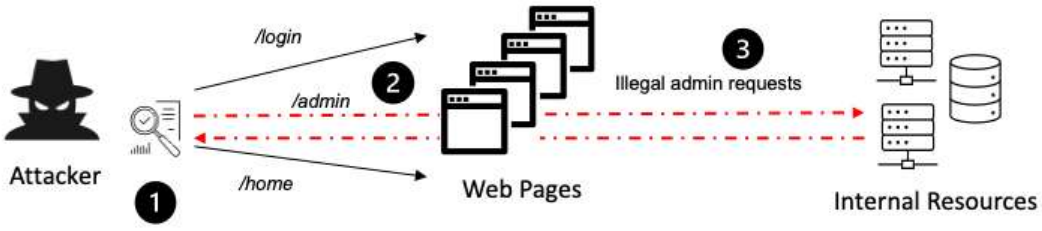

Figure 2.1 : Broken access control attack scenario

2.1.1.2 Cryptographic Failures

Prevalence and Evolution Cryptographic Failures has dropped two positions to #4 in the OWASP Top 10:2025, affecting 3.80% of applications on average with over 1.6 million occurrences in contributed data. This category encompasses 32 CWEs, including CWE-327 (Use of a Broken or Risky Cryptographic Algorithm) and CWE-326 (Inadequate Encryption Strength), reflecting a shift from its prior higher ranking due to improved API adoption but persistent misuse(Andrew van der Stock et al., 2025).

Key Vulnerabilities Common issues include weak algorithms like MD5 or SHA-1, cleartext transmission of sensitive data (CWE-319), hard-coded keys (CWE-321), and improper key management such as reusing nonces or expired keys. Scenarios involve unencrypted data at rest or in transit, predictable randomness (CWE-330), and misuse of encryption to bypass access controls rather than complement them.

Prevention Strategies Prioritize modern algorithms (e.g., AES-256, SHA-256), enforce TLS 1.3 for transit, use secure key derivation like Argon2 or PBKDF2 with salts, and implement proper randomness from approved sources. Rotate keys regularly, avoid hard-coding, and validate cryptographic operations through code reviews and automated tools.

Research Gaps and Implications Despite detailed CWE mappings, limited empirical data exists on cloud-native cryptographic failures in serverless environments or AI-driven key generation. Future theses could investigate adoption barriers in Malaysian fintech, linking cultural risk perceptions to implementation gaps(Andrew van der Stock et al., 2025).

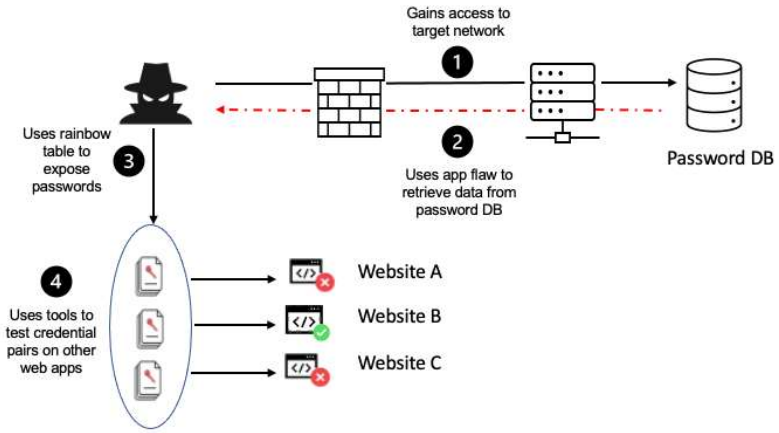

Figure 2.2 : Cryptographic failures attack scenario

2.1.1.3 Injection

Prevalence and Evolution Injection has fallen two spots from #3 to #5 in the OWASP Top 10:2025, with an average incidence rate of 3.08% across 100% tested applications and 1,404,249 total occurrences. Mapped to 37 CWEs, it boasts the highest number of CVEs at 62,445, driven by high-frequency XSS alongside devastating low-frequency SQLi(Andrew van der Stock et al., 2025).

Key Vulnerabilities Attackers inject malicious data through unsensitized inputs to execute unauthorized commands, spanning SQL/NoSQL injection, OS command injection, LDAP, and expression language variants. Common scenarios include second-order SQLi evading detection, template injection in cloud configs, and XSS persisting via databases for client-side execution.

Prevention Strategies Use positive input validation with whitelists, parameterized queries/stored procedures, and Object Relational Mapping (ORM) frameworks; escape special characters per context. Apply least privilege on databases/OS, use linting tools like ESLint with noeval, and conduct interactive testing with IAST/IAE tools.

Research Gaps and Implications While extensively mapped (37 CWEs), gaps remain in second-order injection detection and emerging AI prompt injection defences. Future theses could explore injection prevalence in Malaysian e-government systems, addressing cultural factors in developer training adoption(Andrew van der Stock et al., 2025).

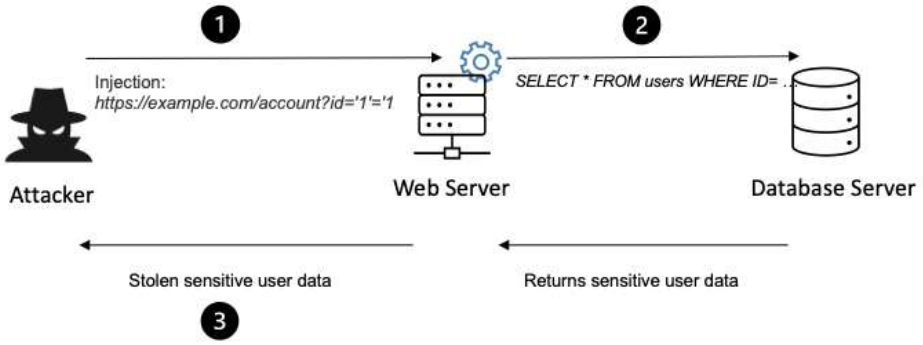

Figure 2.3 : Injection attack scenario

2.1.1.4 Insecure Design

Prevalence and Evolution Insecure Design has slipped two positions from #4 to #6 in the OWASP Top 10:2025, introduced in 2021 with growing industry emphasis on threat modelling showing improvements. It stems from flawed architectural and workflow decisions rather than implementation errors, affecting complex APIs and business logic exposed through modern distributed services(Andrew van der Stock et al., 2025).

Key Vulnerabilities Flaws include missing security controls like explicit authorization, rate limits, and input validation; reliance on trusted user behaviour; and failure to model abuse cases. Real-world examples encompass logic vulnerabilities in recovery/approval flows, flawed AI component assumptions, and insecure design enabling large-scale exploits in APIs and serverless architectures.

Prevention Strategies Establish secure design practices via threat modeling (STRIDE, attack trees), adopt shift-left security with design reviews, and implement core security patterns like secure by default and fail securely. Use automation for design validation, conduct architectural risk analysis, and integrate security champions in development teams.

Research Gaps and Implications Despite progress, empirical data on AI/ML-specific design flaws and cultural influences on threat modeling adoption remains sparse. Future theses could examine insecure design in Malaysian digital services, bridging gaps in localized architectural security practices(Andrew van der Stock et al., 2025).

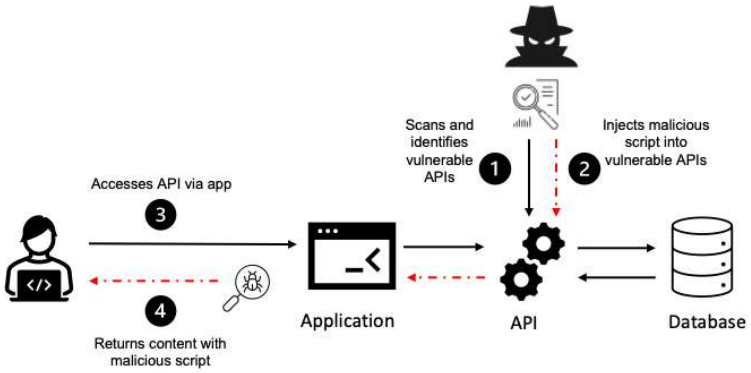

Figure 2.4: Insecure Design attack scenario

2.1.1.5 Security Misconfiguration

Prevalence and Evolution Security Misconfiguration has risen dramatically from #5 to #2 in the OWASP Top 10:2025, with the highest incidence rate at 4.53% average across 99% of applications and 2.5 million occurrences. Mapped to 24 CWEs including CWE-16 (Configuration), it reflects widespread issues in cloud and container environments(Andrew van der Stock et al., 2025).

Key Vulnerabilities Issues stem from unpatched systems, default credentials, verbose error messages, directory listings, and misconfigured HTTP headers (e.g., missing HSTS). Cloud-specific problems include overly permissive S3 buckets, exposed Kubernetes APIs, and XML External Entity (XXE) parsers; scenarios enable data exposure, DoS via resource exhaustion, and privilege escalation.

Prevention Strategies Automate secure configurations via Infrastructure as Code (IaC) scanning, implement secure defaults without sample data, remove unnecessary features, and apply principle of least privilege. Enable logging/monitoring, conduct automated scans, and segment networks with WAFs for HTTP traffic.

Research Gaps and Implications While cloud misconfigs dominate, gaps persist in automated remediation efficacy for hybrid environments and cultural training impacts. Future theses could analyze misconfigurations in Malaysian SMEs, integrating behavioral factors for better compliance(Andrew van der Stock et al., 2025).

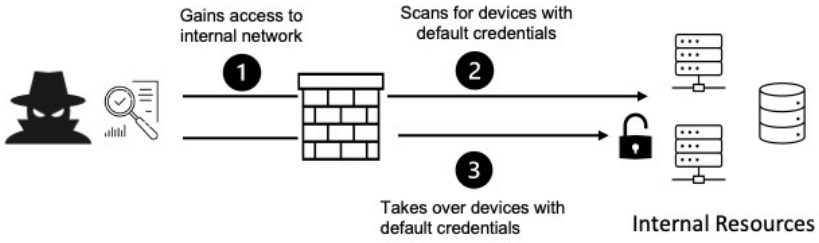

Figure 2.5 : Security Misconfiguration attack scenario

2.1.1.6 Software Supply Chain Failures

Prevalence and Evolution Software Supply Chain Failures has climbed to #3 in the OWASP Top 10:2025, up from #4 in 2021, driven by escalating real-world incidents like SolarWinds and XZ Utils backdoor. It involves 15 CWEs, notably CWE-829 (Inclusion of Functionality from Untrusted Control Sphere), with high-impact low-frequency attacks targeting dependencies and CI/CD pipelines(Andrew van der Stock et al., 2025).

Key Vulnerabilities Risks include untrusted code from repositories, compromised upstream dependencies (e.g., typosquatting), insecure CI/CD pipelines with unsigned artifacts, and lack of SBOMs for visibility. Attacks range from malware injection via npm packages to nation-state supply chain compromises exploiting build systems.

Prevention Strategies Verify upstream integrity with digital signatures and VEX documents, generate/validate SBOMs, and monitor dependencies with SCA tools. Secure pipelines with ephemeral environments, signed commits, and least privilege; conduct supplier assessments and reproducible builds.

Research Gaps and Implications Empirical studies on SBOM adoption effectiveness and open-source governance in diverse regions are limited. Future theses could investigate supply chain risks in Malaysian software ecosystems, addressing cultural procurement practices(Andrew van der Stock et al., 2025).

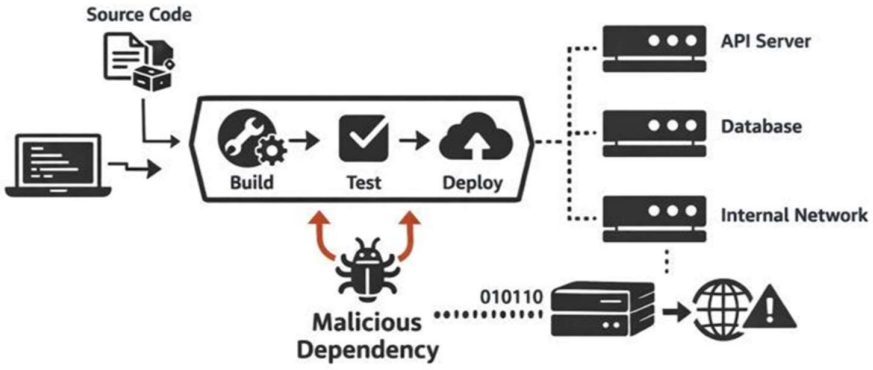

Figure 2.6 : Software Supply Chain Failures attack scenario

2.1.1.7 Authentication Failures

Prevalence and Evolution Authentication Failures has advanced from #8 to #7 in the OWASP Top 10:2025, measured through contributed data with notable incidence in credential stuffing attacks. Encompassing 13 CWEs like CWE-287 (Improper Authentication) and CWE-521 (Weak Password Requirements), it highlights persistent issues despite passwordless trends(Andrew van der Stock et al., 2025).

Key Vulnerabilities Weaknesses include broken password policies, brute-force susceptibility, missing multi-factor authentication (MFA), and session flaws like fixation or predictable tokens. Credential stuffing exploits breached lists, while API auth bypasses occur via weak JWT validation or missing bearer tokens.

Prevention Strategies Implement multi-channel MFA, strong password policies with checkers, and secure credential storage using peppering; detect anomalies with rate limiting and failed login tracking. Use cryptographic tokens for APIs, secure session management, and monitor for stuffing with device fingerprinting.

Research Gaps and Implications Gaps exist in MFA fatigue defenses and behavioral biometrics efficacy across cultures. Future theses could explore authentication in Malaysian mobile banking, incorporating user behavior and cultural trust factors(Andrew van der Stock et al., 2025).

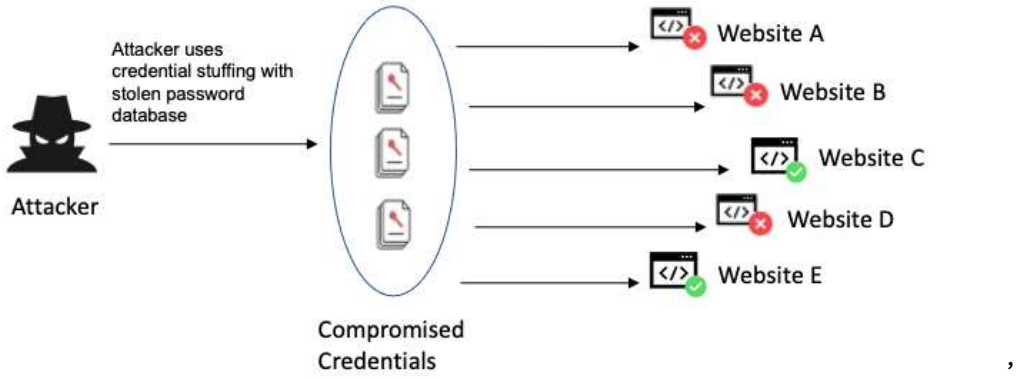

Figure 2.7 : Authentication failures attack scenario

2.1.1.8 Software and Data Integrity Failures

Prevalence and Evolution Software and Data Integrity Failures is a new #8 entry in the OWASP Top 10:2025, merged from 2021's Insecure Deserialization (#25) and Security Logging (#11), with rising concerns over deserialization gadgets and CI/CD risks. It maps to 10 CWEs, including CWE-502 (Deserialization of Untrusted Data), reflecting supply chain and update vulnerabilities(Andrew van der Stock et al., 2025).

Key Vulnerabilities Issues involve unverified software updates (e.g., SolarWinds-style), deserialization exploits via gadgets like ysoserial, and insecure CI/CD pipelines lacking signed artifacts. Data integrity flaws include unsigned JSON/XML and direct object references bypassing validation.

Prevention Strategies Enforce integrity checks with digital signatures, SBOMs, and VEX for updates; use safe serialization (JSON preferred over XML/binary). Secure pipelines with reproducible builds, signed commits, and runtime protections like WAFs for deserialization.

Research Gaps and Implications Limited data on deserialization gadget chains in modern languages and cultural barriers to integrity tooling adoption persist. Future theses could assess risks in Malaysian DevOps practices, linking to compliance and training needs(Andrew van der Stock et al., 2025).

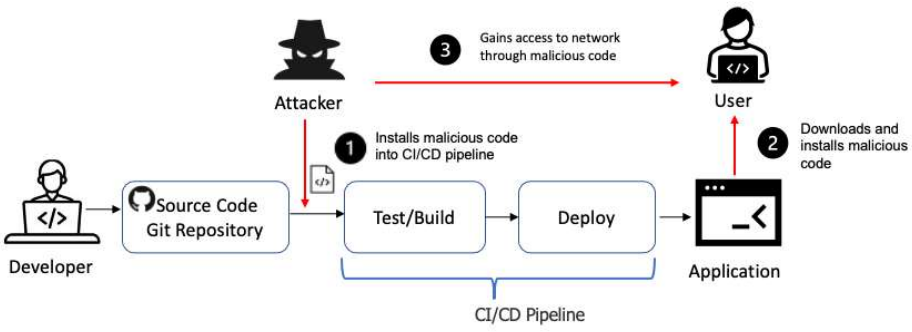

Figure 2.8 : Software and Data Integrity Failures attack scenario

2.1.1.9 Security Logging and Alerting Failures

Prevalence and Evolution Security Logging and Alerting Failures ranks #9 in the OWASP Top 10:2025, a new category from 2021's #11, emphasizing insufficient evidence for security events in detection engineering. It aligns with 9 CWEs like CWE-778 (Insufficient Logging) and CWE-223 (Omission of Security-relevant Information), critical for incident response(Andrew van der Stock et al., 2025).

Key Vulnerabilities Deficiencies include inadequate log detail (e.g., missing auth events, session expiry), excessive sensitive data exposure, and lack of alerting on anomalies like brute force or privilege escalation. Cloud challenges involve siloed logs across services and failure to retain audit trails for forensics.

Prevention Strategies Log all auth/access events with context (who, what, when), avoid sensitive data, and centralize with SIEM; set real-time alerts for high-risk actions. Ensure immutable logs, sufficient storage (1 year+), and parser compatibility for automated analysis.

Research Gaps and Implications Gaps in automated alerting efficacy for AI-driven threats and cultural logging maturity in emerging markets remain. Future theses could evaluate logging in Malaysian critical infrastructure, integrating behavioral and regulatory factors(Andrew van der Stock et al., 2025).

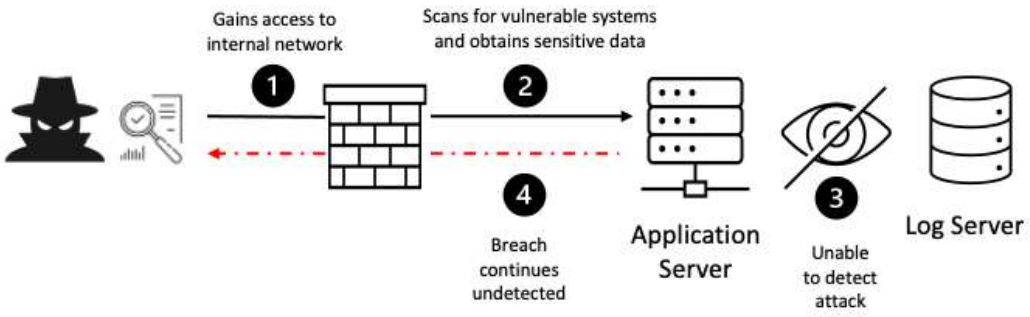

Figure 2.9 : Security Loggin and Alerting failures attack scenario

2.1.1.10 Mishandling of Exceptional Conditions

Prevalence and Evolution Server-Side Request Forgery (SSRF) has dropped from #10 to #10 in the OWASP Top 10:2025 but expanded to encompass Mishandling of Exceptional Conditions, reflecting 2.99% incidence with 1.2 million occurrences. Mapped to 25 CWEs including CWE-918 (SSRF) and CWE-209 (Information Exposure Through Error), it highlights DoS and info leak risks(Andrew van der Stock et al., 2025).

Key Vulnerabilities Traditional SSRF allows internal resource access via redirects/headers; blind variants confirm via timing/oracles. Exceptional conditions expose stack traces, database errors, or config details; DoS arises from resource exhaustion, while RCE exploits deserialization in responses.

Prevention Strategies Apply whitelisting/blacklisting for URLs, disable redirects, and use firewalls; resolve DNS locally without forwarding. Sanitize errors to generic messages, implement fail-safe logic, and monitor for anomalies like high outbound traffic.

Research Gaps and Implications Empirical blind SSRF detection and exceptional handling in microservices lack depth, especially culturally. Future theses could probe SSRF in Malaysian cloud migrations, addressing error disclosure norms(Andrew van der Stock et al., 2025).

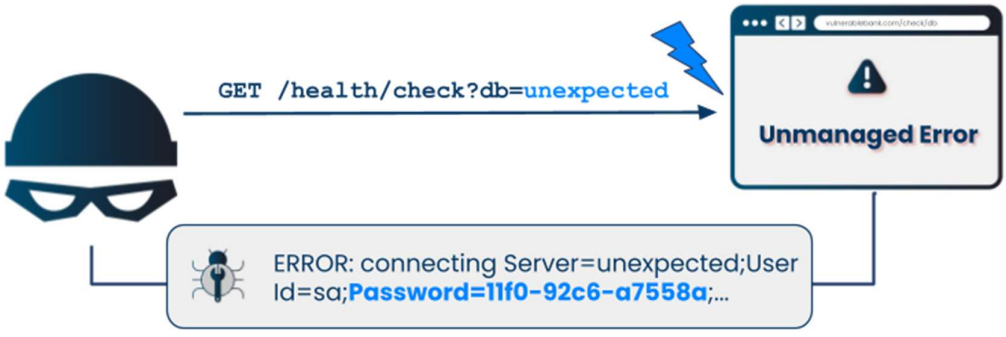

Figure 2.10 : Mishandling of Exceptional Conditions attack scenario

2.1.2 The Rise of AI-Assisted Development (“Vibe Coding”)

The integration of Large Language Models (LLMs) such as GitHub Copilot, ChatGPT, Cursor AI, and Codeium AI into software development has significantly reshaped the coding landscape, enabling substantial productivity gains, automation support, and enhanced debugging capabilities (Haque et al., 2025). In parallel, LLMs have introduced a paradigm shift in Automated Program Repair (APR), where they now play a central role in identifying and generating fixes for software bugs (Puvvadi Sai Kumar Arava Adarsh Santoria et al., 2025). Their emergence has revolutionized APR by enabling systems to understand and generate solutions informed by large-scale contextual reasoning, effectively simulating the decision-making processes of human developers across diverse codebases (Puvvadi Sai Kumar Arava Adarsh Santoria et al., 2025).

LLMs and agent-based coding systems represent an exponential step forward in APR, leveraging broad knowledge of programming languages, reasoning techniques, and tool-calling capabilities to support iterative problem-solving, adaptive learning, and dynamic decision-making (Puvvadi Sai Kumar Arava Adarsh Santoria et al., 2025). This advancement has underpinned the rise of “vibe coding” practices, in which developers increasingly rely on AI-assisted suggestions to accelerate implementation and experimentation, often accepting code snippets with minimal manual verification (Haque et al., 2025). However, this shift also introduces new security and quality concerns, as LLMs are trained primarily on publicly available code, which may embed insecure programming patterns and outdated defensive practices (Haque et al., 2025).

For example, GitHub Copilot has been observed to propagate insecure coding practices by suggesting code snippets that are vulnerable to SQL injection, cross-site scripting (XSS), and other common security flaws, thereby transferring known vulnerabilities into new codebases (Haque et al., 2025). These risks are further exacerbated by the phenomenon of LLM hallucination, which occurs when a model generates information that is incorrect, inconsistent, or nonsensical, undermining the reliability and correctness of the proposed code (Haque et al., 2025). Hallucinated outputs can manifest as syntactically correct but logically flawed code, meaning that

the generated snippets may compile and pass basic tests while still introducing functional errors or security weaknesses (Haque et al., 2025).

This creates a false sense of security, as developers may assume that AI-generated code is both functional and secure, only to discover latent defects or vulnerabilities later in the development lifecycle (Haque et al., 2025). At the same time, current LLMs suffer from limited context windows, which constrain their ability to efficiently process and analyze large codebases consisting of thousands of files (Puvvadi Sai Kumar Arava Adarsh Santoria et al., 2025). As a result, these systems often exhibit accuracy issues in complex edits particularly those spanning multiple lines or files and tend to incur higher error rates due to insufficient context-sensitive code generation mechanisms (Puvvadi Sai Kumar Arava Adarsh Santoria et al., 2025). Together, these limitations highlight the need for more rigorous integration and validation strategies when deploying AI-assisted coding tools in security-sensitive web application development.

The rapid adoption of AI coding agents has underpinned the rise of "vibe coding," a practice where developers rely on AI suggestions and frequently accept code snippets with minimal manual verification. While this accelerates development, it introduces a dangerous cycle of security risks, as LLMs may embed insecure programming patterns derived from their training data. Consequently, developers may operate under a false sense of security, unknowingly introducing latent defects or vulnerabilities such as SQL injection and XSS directly into their codebases. The cyclical nature of this vulnerability propagation is illustrated in Figure 2.11

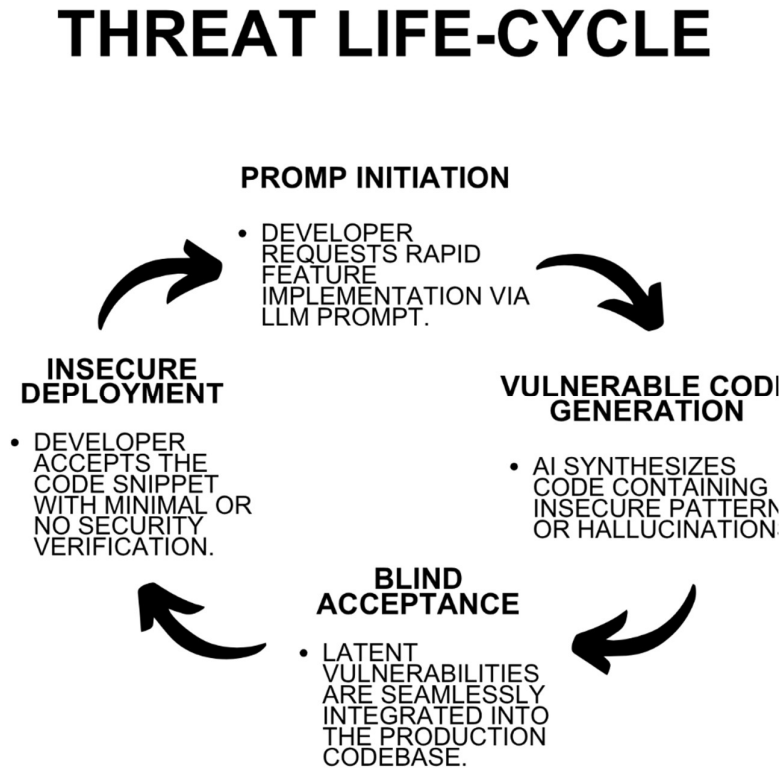

Figure 2.11 : Vibe Coding Threat Life-Cycle

2.1.3 Security Risks and Trust Issues in AI-Generated Code

Empirical evaluations of AI-assisted coding tools reveal significant security concerns in generated code. For instance, a controlled study of GitHub Copilot produced 89 distinct coding scenarios, yielding 1,689 programs, of which approximately 40% were found to contain security vulnerabilities (Pearce et al., 2022). This high vulnerability rate underscores that AI-generated suggestions can directly introduce exploitable weaknesses into real-world codebases, not just in contrived test environments (Pearce et al., 2022). The study attributes this risk to the fact that Copilot is built on a large language model trained on open-source code that includes “public code…with insecure coding patterns,” thereby increasing the likelihood of synthesizing code that incorporates these undesirable security practices (Pearce et al., 2022).

Comparative analyses of human-written and AI-generated code further highlight divergent quality and security profiles. Research indicates that AI-generated code tends to be simpler and more repetitive, yet more prone to unused constructs and hardcoded debugging remnants, whereas human-written code generally exhibits greater structural complexity and a higher prevalence of maintainability-related issues

(Cotroneo et al., 2025). Critically, AI-generated code also contains a higher proportion of high-risk security vulnerabilities compared to human-written code, with the authors noting that AI models produce more vulnerable code in both Python (over 5,000 samples) and Java (over 18,000 samples) while generating fewer lines of code (-6.75) and using significantly fewer tokens (-63.74) (Cotroneo et al., 2025). This suggests reduced structural and logical complexity and lower lexical diversity, which can obscure vulnerabilities and complicate manual review (Cotroneo et al., 2025).

The same study finds that AI models are particularly prone to recurring high-severity issues such as injection flaws, insecure configurations, and information exposure, which closely align with critical OWASP Top 10 categories like Injection and Security Misconfiguration (Cotroneo et al., 2025). These patterns raise concerns about the long-term security posture of applications that heavily rely on AI-generated code, especially when developers treat the suggestions as “correct by default” without rigorous security vetting (Cotroneo et al., 2025). Moreover, broader evaluations of large language models for code emphasize that mis calibrated trust in AI outputs can lead to severe repercussions, including exploitable security vulnerabilities, compromised data integrity, and costly inefficiencies in development and maintenance (Khati, 2025).

Additional trust-related issues include hallucination, where models generate invalid code or invoke non-existent methods, and memorization, in which models may regurgitate sensitive or proprietary code fragments from training data, thereby raising privacy and licensing concerns (Khati, 2025). Such behaviours complicate efforts to assess the trustworthiness and reliability of AI-generated code, as they introduce both functional and ethical risks (Khati, 2025). In particular, the ethical implications of buggy or insecure code suggestions are substantial, as they can introduce latent vulnerabilities that undermine user confidence in AI-assisted development tools and expose organizations to unintended security and compliance risks (Khati, 2025). Together, these findings indicate that integrating AI-generated code into web application development requires stringent review, security-aware training data, and explicit mechanisms to calibrate developer trust.

Comparative analyses of human-written and AI-generated code highlight divergent quality and security profiles. Research indicates that AI models produce more vulnerable code in languages like Python and Java while generating fewer lines of code and using significantly fewer tokens. This reduced structural and logical complexity, along with lower lexical diversity, can obscure vulnerabilities and complicate manual review processes. Table 2.2 summarizes these key empirical differences, emphasizing the heightened security risks associated with unverified AI-generated code

Parameter	Human-Written Code	AI-Generated Code
Structural & Logical Complexity	Higher (greater complexity and maintainability issues)	Lower (simpler, reduced lexical diversity)
High-Risk Security Vulnerabilities	Lower proportion	Significantly higher proportion
Code Repetition & Unused Constructs	Lower	Higher (prone to hardcoded debugging remnants)
Length & Token Usage	Higher (standard verbosity)	Lower (generates fewer lines of code and tokens)

Table 2.2: Comparison of Quality and Security Attributes: Human-Written vs. AI-Generated Code

2.2 Vulnerability Evaluation

2.2.1 Traditional Assessment Techniques (SAST, DAST, Penetration Testing)

Static Application Security Testing (SAST) is a widely used technique for identifying security vulnerabilities in source code during early development stages. SAST tools analyze the application's codebase to evaluate code coverage and perform data-flow testing, path testing, and loop testing, enabling the detection of potential flaws before deployment (Dencheva & Monaghan, 2022). SAST is commonly referred to as white-box testing because the tester has full access to the source code and complete knowledge of the system's architecture, networks, and implementation details (Dencheva & Monaghan, 2022). By examining the code at a granular level, SAST supports "lower-level" testing activities such as unit testing and integration

testing, with the primary aim of checking code quality and enforcing secure coding practices (Dencheva & Monaghan, 2022).

In contrast, Dynamic Application Security Testing (DAST) is typically classified as black-box security testing, where the tester does not have access to the source code or internal system details and interacts with the application through its user interface while it is running (Dencheva & Monaghan, 2022). DAST departs from traditional static analysis by assessing the application during runtime, offering a real-world evaluation of how the system behaves under simulated attack conditions (Singh et al., 2024). The methodology involves actively and controllably probing live applications through simulated cyber-attacks, including the injection of malicious payloads and the emulation of common exploit techniques such as SQL injection and cross-site scripting (Analysis of Web Application Vulnerabilities Using DAST, n.d., p. 1). By observing how the application responds to these simulated threats, DAST helps identify functional vulnerabilities and misconfigurations that may not be evident from static code inspection alone (Singh et al., 2024). DAST is therefore applied at “higher levels” of the software development lifecycle (SDLC), such as system testing and acceptance testing, where the focus shifts from code quality to overall system behavior and security posture (Dencheva & Monaghan, 2022).

A key trade-off between SAST and DAST lies in timing and cost. SAST tools, which operate directly on the source code, generally require less time to scan for security vulnerabilities compared to DAST, and they detect flaws earlier in the SDLC, when remediation tends to be less costly and disruptive (Dencheva & Monaghan, 2022). In contrast, DAST usually identifies vulnerabilities toward the end of the SDLC, when changes to the architecture or functionality may be more expensive to implement (Dencheva & Monaghan, 2022). Beyond automated SAST and DAST, manual web application penetration testing remains a highly effective method for unearthing security weaknesses, but it is often time-consuming and resource-intensive, which limits its scalability for large or fast-paced projects (Altulaihan et al., 2023). To address these limitations, automatic penetration testing frameworks have been proposed, which automate the execution of penetration-test tasks and reduce human effort, thereby improving time-efficiency while maintaining a relatively safe and systematic testing process (Altulaihan et al., 2023).

While both Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) are foundational to traditional security assessments, they operate on fundamentally different paradigms. SAST functions as a white-box testing approach that analyzes raw source code early in the Software Development Life Cycle (SDLC), whereas DAST adopts a black-box methodology, interacting with a running application to identify runtime and configuration flaws. Understanding the distinct capabilities and limitations of each approach is crucial for identifying the existing gaps in automated vulnerability assessment. Table 2.3 summarizes the core operational differences between SAST and DAST.

Feature/Parameter	SAST (Static Analysis)	DAST (Dynamic Analysis)
Testing Approach	White-box(requires full access to source code).	Black-box (simulates external attacker without source code).
SDLC Integration	Early stages(during coding and initial builds).	Late stages (requires a running application/testing environment).
Execution State	Analyzes static, uncompiled, or resting code.	Interacts with the active, running web application.
Vulnerability Coverage	Excellent for coding syntax, hardcoded secrets, and structural flaws.	Excellent for runtime errors, server misconfigurations, and authentication bypasses.
False Positive Rate	High (frequently flags unused or unreachable code as vulnerable).	Lower (requires an actual response from the server to flag an issue).

Table 2.3: Comparison of Static (SAST) and Dynamic (DAST) Application Security Testing

2.2.2 Limitations of Existing Security Assessment Tools

Despite the widespread adoption of automated security assessment tools, current solutions exhibit several critical limitations that constrain their effectiveness in practice. Existing web vulnerability scanners are unable to comprehensively detect all OWASP Top 5 vulnerabilities using a single tool, and no single scanner can reliably identify every type of vulnerability because each tool employs different detection mechanisms and coverage heuristics (Buja et al., 2024). As a result, organizations often need to combine multiple scanners a practice that increases operational complexity and may still leave gaps in (Buja et al., 2024).

A persistent issue across both open-source and commercial scanners is the high rate of false positives, particularly in open-source tools, which tends to generate excessive and sometimes misleading alerts (Buja et al., 2024). Static analysis-based approaches suffer from a similar problem, as they infer context from the code without executing it, which can lead to false positives when the model misinterprets control or data flows (Cruz et al., 2023). These false positives can be especially problematic in legacy systems or projects that were not designed with continuous security analysis in mind, because the lack of contextual metadata further amplifies inaccurate or irrelevant findings (Cruz et al., 2023).

Certain tools also suffer from functional gaps. For example, IronWASP does not support authentication mechanisms, which means it cannot detect vulnerabilities that only become accessible after a user logs into the application (Buja et al., 2024). More broadly, static analysis cannot account for all vulnerability types, particularly those that manifest only during runtime or depend on complex interaction patterns (Cruz et al., 2023). Software Composition Analysis (SCA) tools, which analyze third-party dependencies and dependency graphs, can generate false positives in real-world projects when static inference fails to capture the actual deployment context (Cruz et al., 2023). Moreover, SCA typically does not provide an actual risk assessment but rather flags potentially vulnerable components, which can create substantial technical debt when developers are forced to manually triage hundreds of alerts in large codebases (Cruz et al., 2023).

Dynamic testing approaches also face limitations. Black-box security tests often rely on a general attack strategy that may miss application-specific or subtle

vulnerabilities, and they rarely report the root cause of identified issues, leaving developers without clear guidance on how to remediate them (Cruz et al., 2023). In addition, these scans typically require longer execution times compared to SAST or SCA, making them less suitable for frequent integration into rapid development cycles (Cruz et al., 2023). Collectively, these limitations highlight the need for complementary methodologies such as hybrid SAST-DAST pipelines, contextualized rule tuning, and human-in-the-loop triage to mitigate false positives, improve coverage, and deliver more accurate and actionable security feedback.

2.2.3 False Positives and Alert Fatigue

A major limitation of static analysis (SA) and automated security testing tools is the high rate of false positive (FP) warnings, which significantly undermines their practical utility. Because these tools rely on conservative approximations to over-approximate possible program behaviors, they often generate a large number of warnings that do not correspond to real bugs (Guo et al., 2023). In many cases, the FP rate can range from 30% to as high as 100%, meaning that a substantial proportion of reported issues are non-exploitable or non-existent in practice (Guo et al., 2023).

Faced with this volume of irrelevant alerts, developers are forced to spend considerable effort sifting through FP warnings to identify the few that indicate genuine security flaws (Guo et al., 2023). This process leads to information overload and cognitive strain, reducing the perceived value of static analysis tools over time (Guo et al., 2023). As FP-induced fatigue grows, developers often become impatient and begin to ignore or dismiss SA warnings altogether, sometimes even deactivating the tools entirely, despite their potential to detect serious bugs (Mitigating False Positive Static Analysis Warnings, 2019). Practitioners in industry studies have explicitly reported dissatisfaction with these tools due to the large number of false positives and the resulting high cognitive and operational burden (Guo et al., 2023).

In industrial DevOps environments, automated security testing exacerbates this problem by generating vast volumes of low-quality data, including security findings scattered across different stages of the software development lifecycle (Voggenreiter et al., 2024). Manually triaging and validating these results requires substantial time and coordination from the entire project team, which conflicts with the fast-paced nature of continuous integration and delivery pipelines (Voggenreiter et al., 2024).

Consequently, the management of security findings has become a significant operational bottleneck, especially when teams attempt to integrate security practices into DevOps workflows with comparable efficiency (Voggenreiter et al., 2024). The quality of automated test outputs often demands further investigation before they can be treated as real issues, given well-known shortcomings such as false positives (Voggenreiter et al., 2024). Interview subjects in one case study described the manual effort to manage security findings as “exhaustive,” and team members frequently neglected follow-up tasks, illustrating how alert fatigue directly constrains the effective adoption of automated security tools (Voggenreiter et al., 2024).

2.2.4 The Gap Between Detection and Remediation

Existing vulnerability detection tools often fall short in supporting the actual remediation of identified issues, creating a persistent gap between finding vulnerabilities and fixing them. Developers frequently require tools to explicitly indicate the underlying vulnerability type (for example, via CWE identifiers) associated with the detected code, as this contextual information is essential for understanding the root cause and selecting an appropriate fix (Zhou et al., 2024). Without such precise classification, security alerts remain abstract and harder to prioritize or translate into concrete code changes, prolonging the time-to-remediation and increasing the risk that vulnerabilities remain unpatched.

Detection accuracy also plays a critical role in whether developers are motivated to act on alerts. When vulnerability detection reaches higher precision, developers develop greater confidence in the reliability of the tool’s outputs and are more likely to perform the necessary remediation steps in a timely manner (Zhou et al., 2024). In contrast, low-precision tools that produce many false positives or poorly explained findings tend to be deprioritized or ignored, further widening the gap between detection and remediation. This underscores the need for assessment tools that not only report where vulnerabilities occur but also provide actionable, context-rich guidance that lowers the cognitive and technical burden on developers.

Recent advances in software engineering suggest that large language models (LLMs) can help bridge this gap. LLMs are increasingly being applied in software engineering tasks such as automated program repair, where they support not only fault detection but also the generation of candidate fixes (Zhou et al., 2024). In this context,

the introduction of purpose-built models like vuln-GPT an LLM designed specifically to discover and address software vulnerabilities illustrates the emerging trend of integrating remediation-oriented intelligence into the vulnerability-management pipeline (Zhou et al., 2024). By combining detection with interpretable vulnerability labeling and repair-aware suggestions, such approaches have the potential to transform static findings into clear, developer-friendly remediation paths, thereby reducing the practical disconnect between identification and resolution of security weaknesses.

2.3 Automated Simulation

2.3.1 Overview of Attack Simulation Techniques

Attack simulation techniques play a central role in both understanding and defending against software vulnerabilities. Within automated exploit research, exploits or proofs of concept (PoCs) are treated as oracles that can verify the correctness of generated patches, enabling the development of more robust vulnerability-repair techniques (Bui et al., 2025). Vulnerability exploitation typically demands sophisticated methods to reach an exploitable state, as not every vulnerable condition is trivially triggerable or weaponizable (Bui et al., 2025).

Automated Exploit Generation (AEG) refers to a class of techniques designed to automatically synthesize a working exploit for a given vulnerability (Bui et al., 2025). Many control-flow hijacking-based AEG approaches operate on program binaries and follow a four-stage workflow: (i) identifying the vulnerability, (ii) acquiring runtime information, (iii) generating exploit payloads, and (iv) verifying the exploit's effectiveness (A Systematic Literature Review on Automated Exploit, p. 9). Crash-analysis-based AEG methods take crash information as additional input, extract the execution trace, and attempt to reproduce the crash in symbolic execution mode; if an exploitable state is discovered, a constraint solver is used to derive malicious inputs that can trigger the exploit (Bui et al., 2025). A significant proportion of AEG techniques aim not only to locate vulnerable states but also to confirm that these states are exploitable, thereby ensuring that the generated exploit is practical rather than purely theoretical (Bui et al., 2025).

Beyond low-level exploit generation, broader attack-simulation paradigms such as chaos engineering and DevSecOps-style exercises offer complementary ways to stress-test systems. Large language models (LLMs) can automate the creation of attack–defence scenarios, supporting more robust and proactive security practices across the software lifecycle, especially when integrated with chaos-engineering-style resilience testing (Mohammed et al., 2025). Chaos engineering, in particular, can expose vulnerabilities that standard scanning tools miss, thereby enhancing threat detection and improving system resilience in fast-moving development environments (Mohammed et al., 2025). Additionally, core DevSecOps practices such as regular table-top exercises, strict least-privilege enforcement, and network segmentation help mitigate risks associated with lateral movement and privilege escalation, which are commonly observed in ransomware and targeted attacks (Mohammed et al., 2025). Together, these techniques illustrate how attack simulation can be leveraged not only to probe weaknesses but also to strengthen organizational preparedness and remediation readiness.

The process of Automated Exploit Generation (AEG) transforms theoretical vulnerability detection into actionable security intelligence. By systematically transitioning from static identification to dynamic runtime execution, AEG frameworks can empirically prove the existence of a flaw. This structured approach typically follows a sequential, four-stage pipeline designed to mimic the methodology of a human security analyst. Figure 2.4 illustrates the standard workflow of the AEG process.

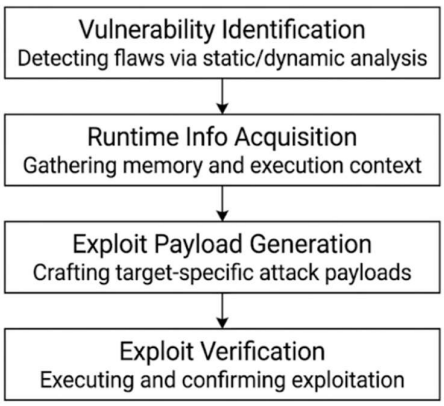

Figure 2.12 : AEG Process Workflow

2.3.2 Headless Browser Automation for Security Testing

Modern security testing increasingly relies on headless browser automation frameworks to simulate realistic user interactions while maintaining efficiency and parallelization. Playwright is a browser automation library that supports parallel test execution and headless mode, enabling testers to run multiple browser sessions without visible UI and thereby improving test speed and scalability (Moń & Pańczyk, 2025). Selenium, another widely adopted framework, operates via the WebDriver protocol and offers a broad ecosystem of tools and libraries for automating web browsers across different platforms (Moń & Pańczyk, 2025). Both Playwright and Cypress, along with Selenium, support the same major web browsers, headless mode, and parallel test execution, making them suitable candidates for integrating dynamic security testing into CI/CD pipelines (Moń & Pańczyk, 2025).

In the context of dynamic application security testing (DAST), headless browsers are often combined with spider and proxy-based testing modes. Spider and proxy modes are two widely used techniques in DAST software: spider mode automatically crawls and explores application pages, while proxy mode sits between the browser and the web application, intercepting and inspecting HTTP(S) traffic generated during browsing (Chiu et al., 2023). Despite ongoing optimizations, spider mode still struggles with complex UI operations that require manual or scripted interaction, such as clicking implicit link elements, performing drag-and-drop actions, or filling out dynamic forms, which can limit the depth and comprehensiveness of coverage (Chiu et al., 2023). In contrast, proxy mode excels at capturing the full sequence of requests and responses, including those generated by user-driven scripts, and can therefore expose more nuanced attack surfaces (Chiu et al., 2023).

Empirical evaluations show that executing test scripts can inject additional interactive elements into webpages, giving the spider more opportunities to explore different page states and thereby increasing code coverage and the number of observed requests (Chiu et al., 2023). Nevertheless, relying solely on spider mode is often insufficient to achieve extensive coverage and generate a high volume of requests, particularly for modern, JavaScript-heavy applications (Chiu et al., 2023). The “Spider-Last” hybrid approach, which combines automated crawling with a final pass of proxy-based interception over scripted or human-driven sessions, has been

shown to outperform either spider or proxy mode used in isolation, offering a more balanced and effective strategy for uncovering security vulnerabilities in dynamic web applications (Chiu et al., 2023).

2.3.3 Visual Proof-of-Concept (PoC) and Exploit Confirmation

Exploit databases play a critical role in turning vulnerability disclosures into concrete, reproducible attack scenarios. These repositories collect Proof of Concepts (PoC) for exploiting specific vulnerabilities, along with supplementary information that describes the preconditions and configurations required to render a system vulnerable (Caturano et al., 2022). This exploit information can be used to programmatically construct vulnerable application environments without manual setup, enabling consistent and repeatable testing conditions for security research and training (Caturano et al., 2022)

Exploit-DB (EDB) entries typically provide rich exploit metadata, including a vulnerability description, the exploit code (or a step-by-step procedure to reproduce the exploit), and, in some cases, a direct reference to the vulnerable component itself (Caturano et al., 2022). These details facilitate both educational use for illustrating how an attack unfolds and practical security validation, where defenders can verify whether a given configuration is actually exploitable under realistic conditions. By combining textual descriptions with executable payloads, PoCs effectively serve as visual and operational proofs that make abstract vulnerabilities tangible and measurable.

In automated platforms such as ExploitWP2Docker, the PoC is treated as a structured input for environment generation. The process involves analyzing the Proof of Concept to extract information about the components required to instantiate a vulnerable environment, such as software versions, plugins, and configuration settings (Caturano et al., 2022). The system attempts to infer the vulnerable software version either from the exploit title or by parsing the PoC text, which allows the framework to automatically assemble Docker-based WordPress instances that reproduce the original exploit scenario (Caturano et al., 2022). This approach not only simplifies exploit confirmation but also supports scalable, repeatable cyber-range exercises where visual PoCs directly drive the creation and validation of vulnerable web environments.

2.3.4 Theoretical Framework of the Simulation Stack (Python, FastAPI)

The simulation stack in this study is conceptually grounded on Python as a general-purpose scripting and orchestration language, combined with FastAPI as the core API-serving framework for the security-testing backend. FastAPI is exceptionally fast due to its foundation on the ASGI (Asynchronous Server Gateway Interface) specification and its native support for asynchronous programming, which enables the handling of many concurrent requests without blocking the event loop (Bednarz & Miłosz, 2025). This asynchronous, event-driven architecture makes FastAPI particularly effective for building microservices and high-performance applications, including real-time data processing and security-testing APIs that must respond quickly to simulated attack traffic (Bednarz & Miłosz, 2025).

In throughput-oriented benchmarks, FastAPI has been shown to achieve, on average, about 80% higher performance than Flask and nearly three times that of Django, highlighting its efficiency in serving high-volume request loads (Bednarz & Miłosz, 2025). This performance advantage is attributed to FastAPI's lightweight design, low-overhead request handling, and the use of relatively simple data models that reduce serialization and processing overhead (Bednarz & Miłosz, 2025). When maximum performance and low latency are critical as is often the case for high-throughput security-testing and simulation systems with relatively straightforward business logic FastAPI is considered a strong framework choice (Bednarz & Miłosz, 2025).

Furthermore, FastAPI's asynchronous model and minimal runtime overhead make it especially well-suited for modern, scalable RESTful APIs, which aligns naturally with the requirements of a security-automation stack that must coordinate test execution, orchestrate headless browsers, and aggregate vulnerability findings in real time (Bednarz & Miłosz, 2025). From a theoretical standpoint, this positions FastAPI as a foundational layer that connects the higher-level simulation logic (written in Python) with the lower-level security-testing tools, ensuring that the simulation stack can sustain high concurrency while maintaining low response latency and predictable resource usage.

2.4 Real-Time Intelligence

2.4.1 Definition of Real-Time Threat Intelligence

Real-time threat intelligence (RTI) is conventionally understood as an extension of cyber threat intelligence (CTI) that emphasizes continuous, near-immediate collection, analysis, and dissemination of security-relevant data. Cyber threat intelligence itself is defined as evidence-based knowledge about existing or emerging threats, including context, mechanisms, indicators, implications, and actionable advice that can inform decisions regarding how an organization should respond to specific menaces or hazards (Sun et al., 2023). In practice, CTI refers to the set of data collected, assessed, and applied concerning security threats, threat actors, exploits, malware, vulnerabilities, and compromise indicators (Sun et al., 2023)

From a theoretical standpoint, CTI entails the systematic acquisition of information intended to proactively safeguard an organization's digital infrastructure and assets (Balasubramanian et al., 2025). The broader CTI ecosystem encompasses a wide range of vendors and tools that collect, analyze, and distribute information on possible cyber threats and vulnerabilities, creating a fragmented but rich network of sources (Balasubramanian et al., 2025). In this pipeline, the input consists of raw cybersecurity data, while the output is transformed knowledge that supports future decision-making and proactive defense strategies, including plans to limit the impact and prevent future attacks (Sun et al., 2023).

Real-time threat intelligence specifically arises when this CTI pipeline is optimized for speed and continuity, so that threat-related information is gathered, processed, and shared with minimal latency (Balasubramanian et al., 2025). Under such conditions, organizations can observe cyber risks in near-real-time, better understand their attackers, respond more quickly to incidents, and anticipate likely next-steps of threat actors (Sun et al., 2023). Recent trends emphasize the management and sharing of multiple types of CTI, as well as their cross-analysis, to address sophisticated, multi-vector cyber attacks (Furumoto et al., 2025). In this context, real-time threat intelligence thus represents a continuous, context-aware flow of actionable intelligence that supports dynamic, proactive security responses rather than slow, periodic reporting and reactive mitigation.

2.4.2 Artificial Intelligence for Security Analysis (OpenAI Integration)

Artificial intelligence has become an increasingly prominent enabler of security analysis, particularly for understanding and hardening AI-generated code and modern web-application traffic. Traditional web application firewalls that rely on hand-crafted rules or classical machine learning require substantial domain expertise and manual tuning, and their detection performance often plateaus when confronted with the growing volume and diversity of unknown web attacks (Alaoui & Nfaoui, 2022). Similarly, anomaly-based machine-learning approaches can detect zero-day attacks by identifying deviations from “normal” behavior, but they suffer from high false-positive rates because defining a stable, context-aware notion of “normality” is inherently difficult in dynamic environments (Alaoui & Nfaoui, 2022).

In contrast, deep learning models have moved to the forefront of vulnerability detection because they can automatically learn meaningful features directly from raw code or network traffic, reducing the need for manual feature engineering (Kumakale et al., 2026). However, these models are often computationally intensive and function as “black boxes,” making their internal decision logic hard to interpret and less suitable for transparent integration into developer workflows (Kumakale et al., 2026). Moreover, the evaluation of deep learning-based web-attack detection models is constrained by the limited availability of high-quality public datasets, many of which are outdated or overly simplistic and fail to capture the complexity of real-world web applications (Alaoui & Nfaoui, 2022).

Classical machine learning models, such as Random Forest, offer a complementary approach that balances accuracy, interpretability, and computational efficiency. In one study on AI-generated Java code, the Random Forest model achieved 95% accuracy and a 0.97 ROC-AUC score in identifying vulnerabilities, demonstrating strong performance while remaining more transparent than deep-learning alternatives (Kumakale et al., 2026). Because these fast, classical models can be executed quickly, they are well-suited for lightweight integration into IDE plugins that warn developers in real time, pre-commit hooks that block unsafe code, and continuous integration/continuous deployment (CI/CD) pipelines that automatically scan pull requests (Kumakale et al., 2026). They also provide a robust baseline for detecting common security flaws, thereby supporting the secure evolution of AI-assisted development practices (Kumakale et al., 2026).

At the same time, machine learning-based security systems face new attack surfaces. ML models are vulnerable to adversarial attacks, where malicious actors manipulate training data or craft inputs that evade detection, for example by contaminating datasets or carefully perturbing features to mislead classifiers (Alaoui & Nfaoui, 2022). A further limitation of traditional ML-based vulnerability detection is that models tend to memorize known vulnerability patterns rather than generalize across broader security concepts, which can reduce their effectiveness against novel or hybrid attack techniques (Kumakale et al., 2026). When integrated with large language models such as OpenAI's models, these AI-powered security components can be used to analyze AI-generated code, generate contextual vulnerability explanations, and suggest remediation patterns, but doing so requires careful calibration to mitigate false positives, adversarial risks, and model opacity.

2.4.3 Simplification of Technical Security Outputs

Automated security tools often produce technically dense outputs that are difficult for many developers to interpret and act upon. Many static application security testing (SAST) tools issue generic warning messages that lack sufficient contextual detail, leading developers to misunderstand or overlook critical findings (Johnson et al., 2026). In practice, developers frequently complain that these messages are too abstract and request clearer, blog-style explanations that walk through the specific cause, impact, and mitigation of each vulnerability (Johnson et al., 2026).

Recent advances in Large Language Models (LLMs) offer a promising avenue for addressing these explainability and usability challenges. LLMs can understand code and generate fluent natural-language descriptions, which makes them suitable for translating raw SAST findings into more accessible, human-readable explanations (Johnson et al., 2026). To this end, the study introduces SAFE (Static Analysis Findings Explainer), an IntelliJ IDEA plugin that leverages GPT-4o to explain the causes, impacts, and mitigation strategies of vulnerabilities detected by SAST tools (Johnson et al., 2026) SAFE integrates directly into the IDE, enabling developers to receive contextual, natural-language explanations without leaving their coding environment (Johnson et al., 2026).

Unlike traditional SAST outputs, SAFE generates step-by-step walkthroughs tailored to each specific finding, explicitly referencing the relevant variables and lines of code that contribute to the vulnerability (Johnson et al., 2026). Human participants in the study reported that these AI-generated explanations were more useful especially for novice and intermediate developers than the original SAST warning messages, and they found the explanations more effective in clarifying vulnerability causes, impacts, and mitigation steps (Johnson et al., 2026). By rendering complex data flows and security logic in plain language, SAFE lowers the expertise barrier and makes vulnerability reports accessible to developers with limited software security background (Johnson et al., 2026) In this way, the simplification of technical security outputs via LLMs serves as a critical bridge between automated detection and actionable remediation within modern development workflows.

2.4.4 Automated Remediation Guidance

Generative Artificial Intelligence (GenAI), and in particular Large Language Models (LLMs) trained on code and natural language, has emerged as a key enabler of automated vulnerability remediation. To address the growing volume and complexity of software vulnerabilities, GenAI is increasingly used to automate tasks such as code generation, optimization, and security-bug fixing, thereby reducing manual effort and accelerating the remediation lifecycle (Costa et al., 2026) In these approaches, LLM-based systems generate context-aware patches that are aligned with the surrounding codebase, which can significantly speed up the fixing process while preserving functional constraints (Costa et al., 2026).

In industrial settings, GenAI engines can be deployed as part of a secure, cloud-based infrastructure for example, a dedicated Azure OpenAI tenant that returns vulnerability-specific suggestions and automated corrections for the code (Costa et al., 2026). These engines leverage advanced AI capabilities to infer the root cause of a vulnerability and propose precise, context-sensitive fixes, effectively transforming security alerts into concrete refactoring actions (Costa et al., 2026). From a broader perspective, training transformer-based language models on large datasets of code vulnerabilities makes it feasible to build tools that can automatically repair security bugs, simultaneously mitigating the threat posed by those vulnerabilities and relieving

developers from the repetitive, error-prone task of manual patching (de-Fitero-Dominguez et al., 2024).

Advances in automated program repair (APR) have shifted toward LLM-driven techniques that complement classical heuristic-, constraint-, and pattern-based methods (de-Fitero-Dominguez et al., 2024). Recent evaluations of state-of-the-art LLMs on APR tasks across Java, Python, and JavaScript show that LLMs can outperform traditional APR approaches, especially when the repair considers code context beyond the immediate buggy line and when larger models are used (de-Fitero-Dominguez et al., 2024). Building on these results, newer work explores the capacity of advanced open-source models such as Code Llama and Mistral to automate the repair of source-code vulnerabilities in lower-level languages like C and C++, which presents additional challenges due to memory-safety and pointer-related flaws (de-Fitero-Dominguez et al., 2024).

Overall, this line of research represents a significant advancement in automated code vulnerability repair, achieved by developing new, efficient methods for generating and applying code modifications at scale (de-Fitero-Dominguez et al., 2024). When integrated into a security workflow, automated remediation guidance powered by GenAI can convert static analysis findings and security scans into ready-made, contextually grounded fixes, thereby reducing time-to-remediation and improving the long-term security posture of software ecosystems.

2.5 Related Works

2.5.1 Comparison of Existing Vulnerability Scanners vs. Simulation Tools

Existing vulnerability scanners and simulation-based security tools differ primarily in their operational scope, depth of analysis, and integration point within the software-development lifecycle. Traditional scanners such as SAST, DAST, and software composition analysis (SCA) tools focus on detecting known vulnerability patterns, misconfigurations, and dependency-related weaknesses by analyzing source code, binaries, or runtime traffic (Altulaihan et al., 2023; Buja et al., 2024; Moń & Pańczyk, 2025). These tools are typically fast, automated, and suitable for integration into CI/CD pipelines, but they often produce high-volume, low-context alerts that require manual triage and may miss logic-driven or interaction-dependent vulnerabilities (Guo et al., 2023; Voggenreiter et al., 2024)

In contrast, simulation-oriented tools and attack-generation frameworks (e.g., automated exploit generation, proxy- and spider-based testing, and chaos-engineering-style platforms) actively probe the system under test, attempting to trigger and demonstrate exploitable behaviors rather than merely flagging suspicious code patterns (Bui et al., 2025; Chiu et al., 2023). By simulating realistic attack scenarios such as injection payloads, authentication bypasses, and lateral-movement flows these tools generate more operationally meaningful findings that reflect actual risk rather than theoretical indicators (Mohammed et al., 2025) However, such approaches are often more resource-intensive and less amenable to fully automated, continuous scanning than traditional scanners.

Empirical evidence suggests that scanners alone are insufficient to achieve comprehensive coverage, particularly for complex, state-ful vulnerabilities that require user-driven interaction or multi-step workflows (Buja et al., 2024; Chiu et al., 2023). Simulation-based tools can enrich purely static or rule-based outputs by validating whether detected vulnerabilities are actually exploitable, but they often operate at a higher level of abstraction and are not designed to generate ready-made remediation code. This dichotomy highlights a need for hybrid approaches that combine the broad, automated coverage of scanners with the exploit-aware, context-rich insights of simulation and attack-generation frameworks.

2.5.2 Analysis of Multi-Agent and AI-Assisted Security Models

Recent advances in AI-assisted development have motivated new security-analysis paradigms centered on multi-agent and collaborative architectures. One representative framework in this space is SCCA (Jiao et al., 2025), which proposes a novel, agent-based model tailored for environments where developers rely heavily on large language models (LLMs) for code generation (Jiao et al., 2025). SCCA aims to bridge the limitations of traditional security tools by orchestrating multiple specialized agents that operate in parallel yet cooperate to produce richer, context-aware outputs (Jiao et al., 2025).

The SCCA framework combines three core agents: (i) an AST-based structural-analysis agent that inspects syntactic and architectural features of the code, (ii) an LLM-enhanced vulnerability-detection agent that leverages natural-language and code-context understanding to identify potential security flaws, and (iii) a data-flow

security-assessment agent that traces sensitive information through execution paths (Jiao et al., 2025). Each agent contributes distinct analytical capabilities, and their results are aggregated and synthesized by a central orchestration layer that manages execution order, data flow, and report generation (Jiao et al., 2025). This collaborative-specialization pattern enables SCCA to produce multi-dimensional security reports that combine syntactic, semantic, and information-flow insights, thereby surpassing the coverage and depth of any single-technique approach (Jiao et al., 2025).

Evaluation results show that the choice of underlying LLM significantly affects both detection accuracy and the quality of vulnerability explanations, with SCCA configured with Claude-4 yielding superior performance compared to baseline configurations (Jiao et al., 2025). The multi-agent design is also shown to outperform traditional single-tool analyses in terms of vulnerability detection rate and remediation-oriented reporting, while still meeting the efficiency and precision constraints required for practical deployment in AI-assisted development workflows (Jiao et al., 2025). These findings align with broader trends in multi-agent security frameworks such as AutoSafeCoder and Agent-based code-security platforms which likewise use coordinated agents to combine static analysis, fuzzing, and large-language-model reasoning for securing AI-generated code (Jiao et al., 2025).

In conceptual terms, these multi-agent models represent a shift from monolithic, single-technique scanners toward modular, cognitively distributed architectures in which different agents specialize in different aspects of security understanding code structure, contextual semantics, and runtime behavior while sharing a common representation of the codebase and the threat landscape (Jiao et al., 2025). This modularity allows the framework to adapt to evolving AI-assisted coding practices, incorporate new LLM models, and scale across different programming languages without requiring a complete redesign of the underlying analysis engine (Jiao et al., 2025). As such, multi-agent AI-assisted security models not only improve detection and explanation but also provide a foundation for integrating automated remediation guidance directly into the developer’s workflow.

2.5.3 Identification of Research Gap

The reviewed literature reveals several well-studied dimensions of web-application and AI-assisted security: automated vulnerability detection (via SAST, DAST, and SCA), attack simulation and exploit generation, headless-browser-based testing, security-aware LLMs, and multi-agent security analysis. Nevertheless, a clear research gap persists in the integration of these capabilities into a unified, simulation-driven security pipeline that connects traditional vulnerability scanners, AI-assisted code generation, and real-time threat-intelligence-informed attack simulation.

On one side, existing vulnerability scanners and multi-agent frameworks such as SCCA provide powerful, context-rich analysis of AI-generated code, but they remain largely request-centric and static in nature, focusing on source-code-level findings rather than on dynamic, attack-driven validation of those vulnerabilities (Jiao et al., 2025). On the other side, simulation and attack-generation tools such as automated exploit-generators, proxy- and spider-based DAST modes, and chaos-engineering-style platforms can verify exploitability but rarely interface directly with vulnerability-scanner outputs or AI-coded sources to propose or validate concrete remediation patches (Bui et al., 2025; Mohammed et al., 2025).

Furthermore, current approaches to AI-assisted remediation and security-explanation tools, including IDE plugins such as SAFE and GenAI-powered patching systems, typically operate in isolation from runtime attack simulation and dynamic threat-intelligence feeds (Costa et al., 2026; Johnson et al., 2026). There is, therefore, a lack of holistic frameworks that simultaneously: (i) detect and explain vulnerabilities in AI-assisted code using multi-agent, LLM-enhanced analysis; (ii) validate detected issues through headless-browser-driven, exploit-aware simulation and attack scenarios; and (iii) generate or recommend automated remediation strategies that are informed by real-time threat-intelligence and exploit-PoC databases. This dissertation addresses that gap by proposing a simulation-driven security-analysis stack that tightly couples vulnerability-scanning outputs, multi-agent AI-assisted analysis, and dynamic attack simulation, thereby closing the loop between detection, simulation-based validation, and automated remediation in modern web-application development.

This dichotomy highlights a need for hybrid approaches that combine the broad, automated coverage of scanners with the exploit-aware, context-rich insights of simulation and attack-generation frameworks. To clearly illustrate these systemic limitations and justify the need for a unified approach, Table 2.4 provides a direct feature comparison between traditional assessment techniques, emerging AI models, and the proposed hybrid solution in this research.

Feature/ Capability	**Traditional SAST**	**Traditional DAST**	**Multi-Agent AI(e.g.,SCCA)**	**VERITAS (Proposed)**
Automated Vulnerability Detection	✔	✔	✔	✔
Runtime Attack Simulation	✘	✔	✘	✔
Visual/Empirical Exploit Verification (PoC)	✘	✘	✘	✔
LLM-Enhanced Contextual Explanation	✘	✘	✔	✔
Automated Remediation Code Generation	✘	✘	Partial	✔

Table 2.4: Feature Comparison of Existing Security Scanners, AI Models, and the Proposed VERITAS System

2.6 Summary

2.6.1 Summary of Key Findings

This literature review establishes that modern web-application security is shaped by a confluence of automated vulnerability scanners, AI-assisted development practices, and increasingly sophisticated attack-simulation techniques. Traditional assessment tools such as SAST and DAST provide broad coverage of common vulnerabilities but are limited by high false-positive rates, incomplete coverage of complex logic flaws, and limited contextual support for remediation (Buja et al., 2024; Dencheva & Monaghan, 2022; Guo et al., 2023)). Attack-simulation methods including automated exploit generation, spider/proxy-mode testing, and chaos-engineering-style approaches add realism by demonstrating whether identified vulnerabilities are actually exploitable, yet they rarely integrate directly with vulnerability-scanner outputs or AI-generated codebases (Bui et al., 2025; Chiu et al., 2023; Mohammed et al., 2025).

At the same time, the rise of AI-assisted development has introduced new capabilities and risks. Large language models (LLMs) enhance coding productivity and automated program repair, but they also propagate insecure coding patterns and hallucinated, syntactically correct yet logically flawed code that can undermine security (Cotroneo et al., 2025; Haque et al., 2025; Khati, 2025). Studies show that AI-generated code tends to contain more high-risk vulnerabilities and simpler, repetitive structures, which makes it both easier to scan and more prone to subtle security weaknesses (Cotroneo et al., 2025; Pearce et al., 2022). Meanwhile, multi-agent security frameworks such as SCCA demonstrate that combining AST-based structural analysis, LLM-enhanced detection, and data-flow security assessment can produce more comprehensive and remediation-oriented reports than any single-technique tool (Jiao et al., 2025)

A critical recurring issue across the literature is the gap between vulnerability detection and remediation. Static and dynamic scanners often produce large volumes of low-context alerts, leading to alert fatigue and underutilization of security tools (Guo et al., 2023). Although AI-assisted remediation and LLM-based explanation tools (e.g., SAFE-style IDE plugins and GenAI-powered fixing engines) can generate contextual explanations and candidate patches, they typically operate in isolation from runtime

attack simulation and real-time threat-intelligence feeds (Costa et al., 2026; de-Fitero-Dominguez et al., 2024; Johnson et al., 2026)

2.6.2 Link to Proposed System (VERITAS)

The key findings above motivate the design and scope of the proposed VERITAS system, a simulation-driven security-analysis framework for AI-assisted web-application development. VERITAS aims to bridge the identified gaps by integrating three core components: (1) traditional vulnerability scanners and LLM-based multi-agent analysis (inspired by frameworks such as SCCA) for detecting and explaining security flaws in AI-generated code, (2) headless-browser-driven attack simulation and exploit-aware DAST modes to validate whether detected vulnerabilities are actually exploitable under realistic conditions, and (3) an automated remediation layer that leverages threat-intelligence-informed PoCs and LLM-powered patch-generation to propose and pre-validate fixes.

By coupling SAST-style code analysis with dynamic, simulation-based testing and AI-assisted explanation, VERITAS moves beyond the static “scan-and-report” model toward a closed-loop security pipeline that connects detection, exploit confirmation, and automated remediation. In particular, the system builds on the multi-agent, SCCA-style architecture to combine AST-based structural analysis, LLM-enhanced vulnerability detection, and data-flow reasoning, while extending this analysis with browser-automation-driven attack scenarios to demonstrate exploitability and reduce false-positive fatigue. At the same time, VERITAS incorporates real-time threat-intelligence and PoC-based exploit data so that remediation suggestions are not only context-aware but also informed by emerging attack patterns observed in the broader ecosystem.

In this way, VERITAS directly addresses the research gap identified in Section 2.5.3: the lack of an integrated, simulation-driven security framework that unifies vulnerability-scanner outputs, AI-assisted code analysis, and dynamic attack simulation into a single, developer-oriented workflow. The literature thus provides both the conceptual foundations and the empirical justification for VERITAS, positioning it as a next-generation tool that leverages AI, multi-agent reasoning, and attack-simulation techniques to improve the security posture of web applications developed under AI-assisted “vibe coding” paradigms.

To encapsulate the breadth of the reviewed literature and visualize the convergence of these distinct research domains, Figure 2.5 presents a taxonomy of the foundational concepts driving this study. This taxonomy highlights how traditional security assessments, attack simulation paradigms, and emerging AI-assisted multi-agent frameworks collectively funnel into the core architectural philosophy of the VERITAS system.

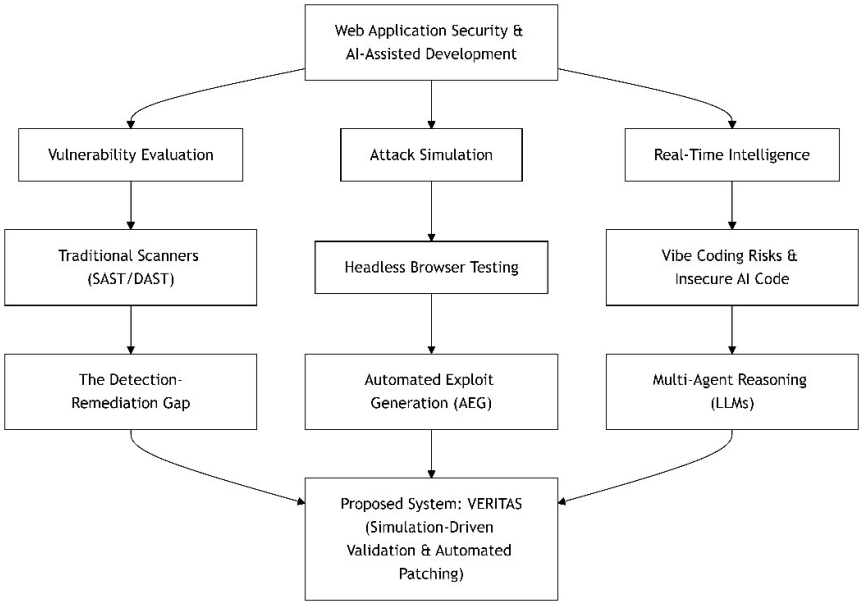

Figure 2.13 : Literature Taxonomy

References

Alaoui, R. L., & Nfaoui, E. H. (2022). Deep Learning for Vulnerability and Attack Detection on Web Applications: A Systematic Literature Review. In *Future Internet* (Vol. 14, Number 4). MDPI. https://doi.org/10.3390/fi14040118

Altulaihan, E. A., Alismail, A., & Frikha, M. (2023). A Survey on Web Application Penetration Testing. In *Electronics (Switzerland)* (Vol. 12, Number 5). MDPI. https://doi.org/10.3390/electronics12051229

Andrew van der Stock, Brian Glas, Neil Smithline, Tanya Janca, & Torsten Gigler. (2025). *The Ten Most Critical Web Application Security Risks*. The OWASP® Foundation, Inc. https://owasp.org/Top10/2025/

Balasubramanian, P., Nazari, S., Kholgh, D. K., Mahmoodi, A., Seby, J., & Kostakos, P. (2025). A cognitive platform for collecting cyber threat intelligence and real-time detection using cloud computing. *Decision Analytics Journal*, *14*. https://doi.org/10.1016/j.dajour.2025.100545

Bednarz, B., & Miłosz, M. (2025). *Benchmarking the performance of Python web frameworks Analiza porównawcza wydajności webowych szkieletów programistycznych w języku Python*.

Bui, Q.-C., Camporese, M., & Tony, C. (2025). *A Systematic Literature Review on Automated Exploit and Security Test Generation* (Vol. 1). https://doi.org/XXXXXXX.XXXXXXX

Buja, A. G., Low, N. N. M. A. A., Zolkeplay, A. F., Azam, N. A., & Isa, F. M. (2024). Analysis of Web Vulnerability Using Open-Source Scanners on Different Types of Small Entrepreneur Web Applications in Malaysia. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, *40*(1), 174–188. https://doi.org/10.37934/araset.40.1.174188

Caturano, F., D'ambrosio, N., Perrone, G., Previdente, L., & Romano, S. Pietro. (2022). ExploitWP2Docker: a Platform for Automating the Generation of Vulnerable WordPress Environments for Cyber Ranges. *International Conference on Electrical, Computer, and Energy Technologies, ICECET 2022*. https://doi.org/10.1109/ICECET55527.2022.9872859

Chiu, K. W., Yang, S. S., Lee, S. J., & Lee, W. T. (2023). An Empirical Evaluation of the Effectiveness of Spider and Proxy Modes for Web Security Testing. *Proceedings - 2023 10th International Conference on Dependable Systems and Their Applications, DSA 2023*, 587–588. https://doi.org/10.1109/DSA59317.2023.00083

Costa, L. A., Fontão, A., Santos, R. P. dos, & Serebrenik, A. (2026). Applying generative artificial intelligence for vulnerability fixing in a proprietary software

ecosystem. *Journal of Systems and Software*, *234*. https://doi.org/10.1016/j.jss.2025.112723

Cotroneo, D., Improta, C., & Liguori, P. (2025). Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity. *Proceedings - International Symposium on Software Reliability Engineering, ISSRE*, 252–263. https://doi.org/10.1109/ISSRE66568.2025.00035

Cruz, D. B., Almeida, J. R., & Oliveira, J. L. (2023). Open Source Solutions for Vulnerability Assessment: A Comparative Analysis. *IEEE Access*, *11*, 100234–100255. https://doi.org/10.1109/ACCESS.2023.3315595

de-Fitero-Dominguez, D., Garcia-Lopez, E., Garcia-Cabot, A., & Martinez-Herraiz, J. J. (2024). Enhanced automated code vulnerability repair using large language models. *Engineering Applications of Artificial Intelligence*, *138*. https://doi.org/10.1016/j.engappai.2024.109291

Dencheva, L., & Monaghan, M. (2022). *Comparative analysis of Static application security testing (SAST) and Dynamic application security testing (DAST) by using open-source web application penetration testing tools MSc Internship MSc in Cyber Security*. https://www.enisa.europa.eu/publications/enisa-threat-landscape-2021

Furumoto, K., Morikawa, T., Kolehmainen, A., Silverajan, B., Takahashi, T., & Inoue, D. (2025). A Comprehensive Survey of Threat Intelligence Research: A Measurement-Based Study. In *ACM Computing Surveys* (Vol. 58, Number 6). Association for Computing Machinery. https://doi.org/10.1145/3772280

Guo, Z., Tan, T., Liu, S., Liu, X., Lai, W., Yang, Y., Li, Y., Chen, L., Dong, W., & Zhou, Y. (2023). Mitigating False Positive Static Analysis Warnings: Progress, Challenges, and Opportunities. *IEEE Transactions on Software Engineering*, *49*(12), 5154–5188. https://doi.org/10.1109/TSE.2023.3329667

Haque, M. A., Siddique, S., Rahman, M. M., Hasan, A. R., Das, L. R., Kamal, M., Sujaee, K., Gupta, K. D., & George, R. (2025). SOK: Exploring Hallucinations and Security Risks in AI-Assisted Software Development with Insights for LLM Deployment. *2025 6th International Conference on Intelligent Data Science Technologies and Applications, IDSTA 2025*, 57–64. https://doi.org/10.1109/IDSTA66210.2025.11202778

Jiao, R., Zhang, Y., Li, J., & Ma, B. (2025). SCCA: A Multi-Agent Code Security Analysis Framework for AI-Assisted Code Generation. *Proceedings - 2025 IEEE 22nd International Conference on Mobile Ad-Hoc and Smart Systems, MASS 2025*, 549–554. https://doi.org/10.1109/MASS66014.2025.00089

Johnson, O., Fomina, A., Krishnamurthy, R., Chaudhari, V., Shanmuganathan, R. K., & Bodden, E. (2026). *Explaining Software Vulnerabilities with Large Language Models*. 194–198. https://doi.org/10.1109/asew67777.2025.00045

Khati, D. (2025). Trustworthiness of Large Language Models for Code. *Proceedings - International Conference on Software Engineering*, 208–210. https://doi.org/10.1109/ICSE-Companion66252.2025.00063

Kumakale, S. S., A, A., Badiger, V. S., Sheetal, Jacintha, A. Ds., & Jagadish, S. (2026). Detection of Vulnerabilities in AI-Generated Java Code Using Random Forest Algorithm. *2026 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, 1–6. https://doi.org/10.1109/IITCEE67948.2026.11394305

Mohammed, K. I., Shanmugam, B., & El-Den, J. (2025). Evolution of DevSecOps and Its Influence on Application Security: A Systematic Literature Review. In *Technologies* (Vol. 13, Number 12). Multidisciplinary Digital Publishing Institute (MDPI). https://doi.org/10.3390/technologies13120548

Moń, M., & Pańczyk, B. (2025). *A comparative analysis of web application test automation tools Analiza porównawcza narzędzi do automatyzacji testów aplikacji internetowych*.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). *Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions*. https://doi.org/10.1109/SP46214.2022.00057

Puvvadi Sai Kumar Arava Adarsh Santoria, M., Machine Learning Manager, S., Jose, S., San Jose, U., Mandi, U., & Sai Prasanna Chennupati Harsha Vardhan Puvvadi, S. (2025). *Coding Agents: A Comprehensive Survey of Automated Bug Fixing Systems and Benchmarks*. 680–686. https://doi.org/10.1109/CSNT.2025.113

Rahman, K., & Izurieta, C. (2022). A Mapping Study of Security Vulnerability Detection Approaches for Web Applications. *Proceedings - 48th Euromicro Conference on Software Engineering and Advanced Applications, SEAA 2022*, 491–494. https://doi.org/10.1109/SEAA56994.2022.00081

Shahid, J., Hameed, M. K., Javed, I. T., Qureshi, K. N., Ali, M., & Crespi, N. (2022). A Comparative Study of Web Application Security Parameters: Current Trends and Future Directions. *Applied Sciences (Switzerland)*, *12*(8). https://doi.org/10.3390/app12084077

Singh, R., Kumar Gupta, M., Patil, D. R., & Maruti Patil, S. (2024). Analysis of Web Application Vulnerabilities using Dynamic Application Security Testing. *2024 IEEE 9th International Conference for Convergence in Technology, I2CT 2024*. https://doi.org/10.1109/I2CT61223.2024.10543484

Sun, N., Ding, M., Jiang, J., Xu, W., Mo, X., Tai, Y., & Zhang, J. (2023). Cyber Threat Intelligence Mining for Proactive Cybersecurity Defense: A Survey and New Perspectives. *IEEE Communications Surveys and Tutorials*, *25*(3), 1748–1774. https://doi.org/10.1109/COMST.2023.3273282

Voggenreiter, M., Angermeir, F., Moyon, F., Schöpp, U., & Bonvin, P. (2024). Automated Security Findings Management: A Case Study in Industrial DevOps. *ACM International Conference Proceeding Series*, 312–322. https://doi.org/10.1145/3639477.3639744

Zhou, X., Zhang, T., & Lo, D. (2024). Large Language Model for Vulnerability Detection: Emerging Results and Future Directions. *Proceedings - International Conference on Software Engineering*, 47–51. https://doi.org/10.1145/3639476.3639762